AN ADVANCED POSTGRESQL FIELD GUIDE

BEYOND THE ROWS.

Advanced **JSONB querying** in PostgreSQL.
From nested documents to meaningful insights.

SQL × JSON × PERFORMANCE

PRESS → TO EXPLORE



51° 30′ 26.4″ N
00° 07′ 39.6″ W

Real-world data doesn't fit in neat boxes.

01 / VARIATION

{ }

Different structures

Laptops have RAM. Headsets have wireless features. Their attributes are not identical.

02 / EVOLUTION

↗

Changing requirements

New properties arrive without needing a new column for every product feature.

03 / INSIGHT



Searchable flexibility

We still need filters, aggregations, and indexes — not just a place to store JSON.

The question: How do we keep flexible documents queryable with SQL?

Same JSON syntax. Different superpowers.

{ } JSON

TEXT REPRESENTATION

- ↗ Preserves input text and whitespace
- ↗ Preserves object key order
- ↗ Reparses for processing
- ↗ Useful when input fidelity matters

VS

{ B } JSONB

OUR FOCUS

- ↗ Stored in decomposed binary form
- ↗ Supports indexing and containment
- ↗ Does not preserve key order / duplicates
- ↗ Usually preferable for querying

Both types validate JSON. JSONB has processing and indexing benefits, with conversion overhead at input.

One table.
Many different shapes.

01_schema.sql — SQL

```sql
CREATE TABLE catalog (
  id   BIGSERIAL PRIMARY KEY,
  data JSONB NOT NULL
);

INSERT INTO catalog (data) VALUES
  ('{"name":"Atlas Pro 14",
     "category":"laptop",
     "price":1299,
     "specs":{"ram":16},
     "tags":["portable","premium"]}');
```

- PostgreSQL-compatible SQL

EXAMPLE ROWS — 06 RECORDS

01	**Atlas Pro 14** Laptop / 16 GB RAM	$1,299
02	**Nova Air** Laptop / 8 GB RAM	$899
03	**Pulse X** Headset / wireless	$149
04	**Forge 16** Laptop / 32 GB RAM	$1,699
05	**Orbit Key** Keyboard / mechanical	$99
06	**Wave Mini** Headset / wireless	$79

Meet your query toolkit.

->

Get JSON

Keep the JSON/JSONB type.

```
data -> 'specs'
```

->>

Get text

Extract a scalar as text.

```
data ->> 'name'
```

@>

Contains

Check document containment.

```
data @> '{...}'
```

?

Key exists

Top-level key or string array element.

```
data ? 'specs'
```

#>>

Path to text

Reach deeply nested values.

```
data #>> '{specs,ram}'
```

@?

JSONPath exists

Match complex nested paths.

```
data @? '$.tags[*]...'
```

Go deep.
Keep your types straight.

DATA / ATLAS PRO 14 JSONB

```
{
  "name": "Atlas Pro 14",
  "specs": {
    "ram": 16
  },
  "price": 1299
}
```

TARGET: specs → ram

STEP 01 / JSON VALUE

```
data -> 'specs' -> 'ram'
```

RETURNS 16 jsonb

STEP 02 / TEXT VALUE

```
data #>> '{specs,ram}'
```

RETURNS "16" text

STEP 03 / NUMERIC COMPARISON

```
(data #>> '{specs,ram}')::int >= 16
```

RETURNS true boolean

Remember: Missing paths typically return SQL NULL. Cast extracted text before numeric comparisons.

Find the signal inside the document.

@> JSONB CONTAINMENT

```
SELECT data ->> 'name' AS product
FROM catalog
WHERE data @>
  '{"category":"laptop"}'::jsonb;
```

3 MATCHES

- Atlas Pro 14
- Nova Air
- Forge 16

? KEY EXISTENCE

```
SELECT data ->> 'name' AS product
FROM catalog
WHERE data ? 'specs';
```

3 MATCHES

- Atlas Pro 14
- Nova Air
- Forge 16

Containment matches structure and values. The ? operator checks only the **top level**.

You set the rules.
SQL finds the rows.

QUERY BUILDER ● LIVE

PRODUCT CATEGORY

Laptops ⌄

MINIMUM RAM

16 GB or more ⌄

MAX PRICE $1,500

$100 $2,000

↺ RESET FILTERS

query_builder.sql COPY SQL ↗

```
SELECT id,
       data->>'name' AS product,
       data->>'category' AS category,
       (data->>'price')::numeric AS price
FROM catalog
WHERE data @> '{"category":"laptop"}'::jsonb
  AND (data #>> '{specs,ram}')::int >= 16
```

QUERY RESULT 1 ROW

ID	PRODUCT	CATEGORY	PRICE
01	Atlas Pro 14	laptop	$1,299

A deterministic teaching simulator running on six in-browser records. Generated SQL can be executed on a PostgreSQL database seeded with sample-data.sql.

When a key isn't enough, follow a pattern.

@? / PATH EXISTS

01

Find products tagged gaming

```
SELECT data ->> 'name'
FROM catalog
WHERE data @?
  '$.tags[*] ? (@ == "gaming")';
```

RESULT **Pulse X · Forge 16**

@@ / PREDICATE CHECK

02

Find products above $1,000

```
SELECT data ->> 'name'
FROM catalog
WHERE data @@ '$.price > 1000';
```

RESULT **Atlas Pro 14 · Forge 16**

✳ **Why it matters:** JSONPath can filter inside arrays and traverse irregular documents. `@?` asks whether a path yields a match; `@@` evaluates a predicate.

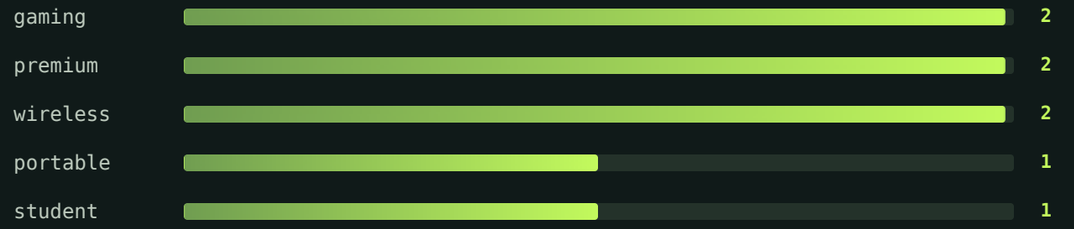
JSON arrays → SQL insight.

top_tags.sql SQL

```sql
SELECT tag, COUNT(*) AS uses
FROM catalog c
CROSS JOIN LATERAL
  jsonb_array_elements_text(
    c.data -> 'tags'
  ) AS tag
GROUP BY tag
ORDER BY uses DESC, tag;
```

Set-returning function + aggregation

MOST COMMON TAGS DEMO DATA



From heterogeneous documents to consistent reporting.

Flexible shouldn't mean impossible to scale.

A / GENERAL PURPOSE

GIN

jsonb_ops

```
CREATE INDEX cat_gin
ON catalog USING GIN (data);
```

Supports `@>`, `?`, `?|`, `?&` and JSONPath search operators.

B / CONTAINMENT OPTIMIZED

GIN

jsonb_path_ops

```
CREATE INDEX cat_path
ON catalog USING GIN
  (data jsonb_path_ops);
```

Focused on `@>`, `@?`, `@@` — not key existence `?`.

C / SCALAR FILTERING

B-tree

expression index

```
CREATE INDEX cat_price
ON catalog
  (((data->>'price')::numeric));
```

Useful for ordered numeric comparisons on extracted scalar values.

Index selection depends on predicates, data distribution, planner estimates, and write/storage costs.

Don't assume it's fast. Measure it.

explain.sql SQL

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT id
FROM catalog
WHERE data @>
  '{"category":"laptop"}';
```

- Run on real data for real measurements

WHAT TO LOOK FOR

Seq Scan
May be cheapest on small tables

OR

Bitmap Index Scan
Can use a suitable GIN index

BUFFERS ACTUAL TIME ROWS

These are illustrative plan shapes, not measured results. PostgreSQL chooses the plan based on estimated cost.

Four pitfalls.
Zero excuses.

01

Text ≠ number

`->>` returns text. Use an explicit safe cast when comparing quantities.

02

Existence is not recursive

`?` searches top-level keys/array elements. For deep search, navigate or use JSONPath.

03

Indexes have a cost

GIN helps reads, but consumes storage and adds write maintenance.

04

Not everything belongs in JSONB

Keep stable identifiers, constraints, and heavily joined relationships relational.

The 30-second challenge.

01 / TYPES — TAP TO REVEAL ↗

Which operator returns text: → or →>?

02 / DEPTH — TAP TO REVEAL ↗

Can ? find a nested key at any depth?

03 / INDEXES — TAP TO REVEAL ↗

Does `jsonb_path_ops` accelerate ??

CLICK ANY CARD TO CHECK THE ANSWER



STRUCTURE MEETS FREEDOM.

01 **Store flexibly.** JSONB brings document-oriented data into PostgreSQL.

02 **Query precisely.** Operators, JSONPath, and SQL analytics unlock nested fields.

03 **Measure wisely.** Pick indexes for your real workload and verify with EXPLAIN.

THANK YOU ↗

REFERENCE DESK

OFFICIAL SOURCES

01 JSON Types & Indexing ↗

02 JSON Functions & Operators ↗

03 Indexes on Expressions ↗

04 Using EXPLAIN ↗

PostgreSQL Global Development Group · Official Documentation